

Alphabet of the Assembler

Categories of Commands

1. Arithmetic and Logical (ALU Commands)
2. Shift Commands
3. Memory Commands
4. Branch Commands
5. Jump and Subroutine Commands
6. Immediate Commands

- **Arithmetic and Logical (ALU Commands)**

OPCODE "0000"	
ADD	000001
SUB	000010
AND	000011
OR	000100
XOR	000101
NOT	000111
OTHER BITS	DON'T CARE

- **Shift Commands**

OPCODE "0001"	
SHL	000001
SHR	000010
ROL	000011
ROR	000100

- **Memory Commands**

OPCODE “0010”	
LOAD	000001
STORE	000010

- **Branch Commands**

OPCODE “0011”	
BEQ	000001
BNE	000010
BLT	000011
BGT	000100

- **Jump and Subroutine Commands**

OPCODE “0100”	
JMP	000001
CALL	000010
RET	000011

- **Comparison commands**

OPCODE “0101”	
CMP	000001

- **Immediate Commands**

OPCODE “0110”	
MOVI	000001

OPERANDS

Operands include registers, immediate values, and memory addresses.

- **REGISTERS**

Symbol: R0, R1,..., R27 (32 registers, encoded as 5-bit values).

BINARY ENCODINGS

GENERAL PURPOSE	
R0	00000
R1	00001
R2	00010
R....	
R27	11011

SPECIAL PURPOSE	
PC (R28)	11100
R-MEM (R29)	11101
LR (R30)	11110
FR (R31)	11111

SYNTAX RULES

GENERAL SYNTAX:

COMMAND OPERAND1, OPERAND2, OPERAND3
 COMMAND OPERAND1, OPERAND3

- **COMMAND:** The operation mnemonic (e.g., ADD, BEQ, MOVI).
- **OPERAND1:** Always register (source 1).
- **OPERAND2:** Immediate value, Register or Mem Address (source 2).
- **OPERAND3:** Destination (Reg or Mem Address)

SINGLE ENDED COMMANDS:

- **NOT R1, R2** (Invert R1 and store result in R2)

- **RET** (Return from subroutine).

BRANCH AND JUMP COMMANDS:

- **JMP** LABEL_NAME
- **BEQ** LABEL_NAME (based on **CMP** command results)

IMMEDIATE COMMANDS:

- **MOVI** R1, #10 (Move 10 into R1).

LABEL DEFINITION:

- **LABEL_NAME:** (Define a label for a memory address).

IMMEDIATE AND MEM VALUES VALUES:

- Before the immediate value there is #
Example #20 immediate value
- Mem address must be between "[]" symbols
 - Example LOAD R1, [0x1000]

COMMENTS:

Comments start with a semicolon (;) and are ignored by the assembler.

Example`

; Program example

MOVI R1, #5 ; Load 5 into R1

Parsing Rules

Line Structure:

- A line can contain:
 1. A label (LABEL:).
 2. An instruction (COMMAND OPERAND1, OPERAND2, OPERAND3).
 3. A comment (;).

EXAMPLE:

- `START:` `// label`
 `; This is the beginning of the program` `// comment`
 `ADD R1, R2, R3` `// command`
 `CMP R1,R2`

Whitespace:

- Lines may include spaces or tabs for alignment.
- Extra whitespace should be ignored.

Case Insensitivity:

- Commands and operands should be case-insensitive (ADD is equivalent to add).

Example Assembly Code:

`; Program to add two numbers`

`START: ; label`

`MOVI R1, #10 ; Load 10 into R1`

`MOVI R2, #20 ; Load 20 into R2`

`ADD R1, R2, R3 ; R3 = R1 + R2`

`STORE R3, MEM[0x0100] ; Save result in memory at 0x0100`

`JMP START ; Repeat without condition`

`struct Instruction`

`{`

`std::string istr_type // ALU SHIFT MEM and so on – 4 bit`

`std::string opcode; // Command mnemonic, e.g., ADD, SUB – 6bit`

`std::string source1; // Source 1 operand (always register) – 5 bit`

`std::string source_2_type; // can be register immediate or mem address – 2 bit`

```
std::string source2; // Source 2 operand (register/immediate/memory) – 16 bit(if reg  
5LSBs)
```

```
std::string dest_type; // Destination type (reg, mem) - 1bit
```

```
std::string destination; // Destination operand (register/memory) 16 bit if reg use 5LSBS
```

```
bool requires_simplification = false; // Flag for memory operations requiring LOAD };
```